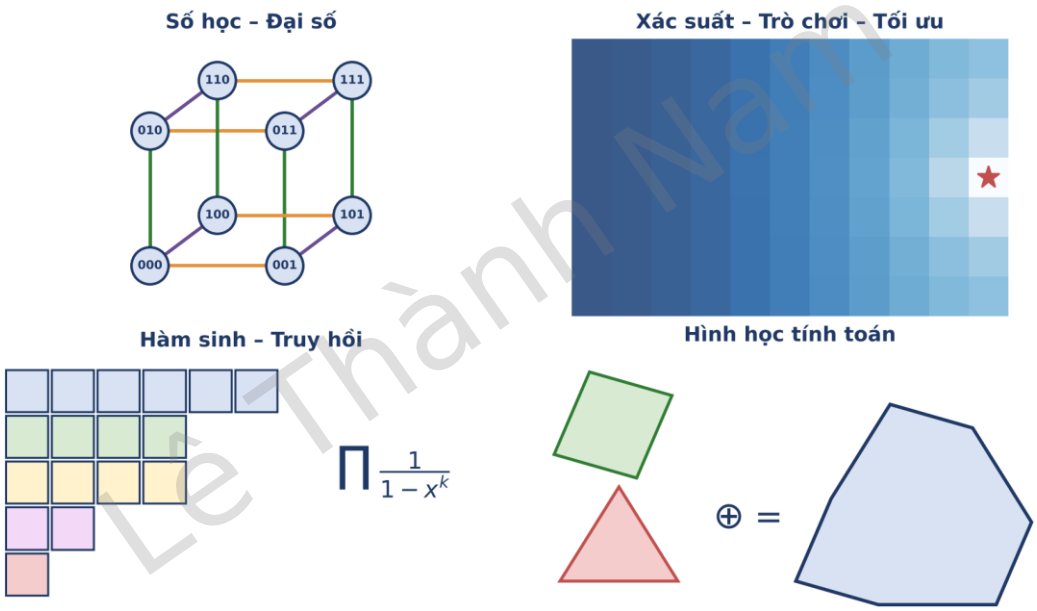
Tổng hợp và biên soạn: Lê Thành Nam

TÀI LIỆU BỒI DƯỠNG HỌC SINH GIỎI TIN HỌC

TOÁN HỌC TRONG TIN HỌC

Chuyên đề nâng cao ôn thi học sinh giỏi Quốc gia (VOI) – Quyển 2

TẬP 1 · LÝ THUYẾT



Lê Thành Nam

Chương 1. SỐ HỌC VÀ ĐẠI SỐ NÂNG CAO
Chương 2. XÁC SUẤT – TRÒ CHƠI – TỐI ƯU
Chương 3. HÌNH HỌC TÍNH TOÁN NÂNG CAO

LỜI NÓI ĐẦU

Quyển 1 của bộ "Toán học trong Tin học – Nâng cao" đã trang bị phần cốt lõi hay gặp nhất trong đề thi chọn học sinh giỏi Quốc gia: sàng và hàm nhân tính, Euclid mở rộng và CRT, Möbius, Pollard rho, Lucas, Burnside, kì vọng, Sprague–Grundy, lũy thừa ma trận, khử Gauss, FFT, bao lồi và đường quét. Quyển 2 này đi tiếp vào những mảng **mà Quyển 1 chưa có bài riêng** nhưng xuất hiện ngày càng nhiều ở câu phân loại của đề VOI và đề chọn đội tuyển.

Cách trình bày giữ nguyên: mỗi bài học xuất phát từ một bài toán có giới hạn kiểu VOI. Ta bắt đầu bằng cách làm mà một học sinh đã học xong Quyển 1 sẽ nghĩ ra, đếm chính xác số phép tính để thấy nó dừng ở đâu, rồi dựng công cụ mới từng bước, kèm phần **vì sao đúng** và phần chạy tay có số liệu cụ thể.

Bộ sách gồm ba tập đi cùng nhau:

• **Tập 1: Lý thuyết** (cuốn sách này) gồm 10 bài học chia thành 3 chương: Số học và Đại số nâng cao; Xác suất – Trò chơi – Tối ưu; Hình học tính toán nâng cao.

• **Tập 2: Bài tập** gồm 30 bài hoàn toàn mới, mỗi bài học có ba bài ở ba mức (★, ★★, ★★★ theo mặt bằng VOI), có subtask như đề thi thật.

• **Tập 3: Hướng dẫn giải** phân tích từng bài từ cách ngây thơ tới cách tối ưu, kèm chương trình và bảng điểm đo thật của từng cách làm.

Mọi chương trình trong sách đều đã được biên dịch theo chuẩn C++14, chạy thử, và đối chiếu với lời giải vét cạn trên nhiều bộ dữ liệu ngẫu nhiên. Mọi con số trong các bảng chạy tay đều được kiểm tra lại bằng máy. Chương trình đọc và ghi tệp theo kiểu "có tệp thì đọc tệp" để dùng được với cả Themis lẫn các trang chấm trực tuyến.

Quyển 2 giả định em đã học Quyển 1. Khi cần, sách nhắc lại ngắn gọn kiến thức cũ và ghi rõ "xem Quyển 1, Bài ...".

Chúc các em học tập say mê và đạt kết quả cao!

Lê Thành Nam

CÁCH HỌC VỚI TÀI LIỆU NÀY

Mỗi bài học đi theo cùng một con đường:

Phần	Nội dung
Khởi động	Một câu hỏi nhỏ, tính được bằng tay, chứa sẵn mầm của ý tưởng chính.
Bài toán	Đề bài cụ thể với giới hạn kiểu VOI, có ví dụ.
Ý tưởng ngây thơ	Cách làm của một học sinh đã học Quyển 1, kèm phần đếm số phép tính.
Bế tắc	Chỉ ra chính xác chỗ cách làm hiện tại không vượt qua được.
Phá bế tắc	Từng bước: quan sát, công thức, chứng minh, chạy tay, chương trình.
Mở rộng	Biến thể và các kĩ thuật đi kèm thường gặp trong đề VOI.
Bẫy thường gặp	Những lỗi làm mất điểm ở mức này: tràn số, sai trường hợp biên, sai số thực.
Ghi nhớ	Tóm tắt cốt lõi.
Luyện tập	Liên hệ tới ba bài tập ở Tập 2 và vài câu tự kiểm tra có đáp số.

Các khung màu trong sách có ý nghĩa cố định:

Khung	Ý nghĩa
KHỞI ĐỘNG	Câu hỏi mở đầu bài học.
BÀI TOÁN	Đề bài chính thức.
CÔNG THỨC	Công thức, định lí hoặc thuật toán cốt lõi.
ĐẾM SỐ PHÉP TÍNH	Phân tích độ phức tạp và bộ nhớ.
BẾ TẮC	Chỗ cách làm hiện tại không vượt qua được.
VÌ SAO ĐÚNG?	Chứng minh tính đúng đắn.
BẪY THƯỜNG GẶP	Các lỗi hay gặp.
GHI NHỚ	Tóm tắt cuối bài.

Lê Thành Nam

Quy ước kí hiệu. $a \mid b$ nghĩa là a chia hết b. $\lfloor x \rfloor$ là phần nguyên dưới của x. Phép XOR theo bit được viết là $\oplus$ trong lời văn và `^` trong chương trình; phép AND, OR theo bit viết là AND, OR. Với mặt nạ bit S và T, "$T \subseteq S$" nghĩa là mọi bit 1 của T cũng là bit 1 của S. $[x^k]\,F(x)$ là hệ số của x^k trong chuỗi lũy thừa $F(x)$. $\pi(n)$ là số số nguyên tố không vượt quá n; φ là hàm Euler, μ là hàm Möbius. Khi đề yêu cầu in số hữu tỉ P/Q "theo modulo p", ta in $P \cdot Q^{-1} \bmod p$.

Bảng tra nhanh: ràng buộc gợi ý độ phức tạp (khoảng 10^8 phép tính đơn giản mỗi giây):

Ràng buộc	Độ phức tạp cần đạt	Ví dụ trong sách
$n \le 20$ (bit)	$O(2^n \cdot n)$	SOS, FWHT (Bài 2)
$n \le 5\,000$	$O(n^2)$	Tối ưu Knuth (Bài 8)
$n \le 2 \cdot 10^5$	$O(n \log n)$	Giao nửa mặt phẳng (Bài 9)
$n \le 10^{11}$	$O(n^{3/4})$, $O(n^{2/3})$	Lucy, sàng Đỗ Giáo (Bài 1)
$n \le 10^{18}$	$O(d^2 \log n)$	Kitamasa (Bài 5), nội suy (Bài 4)

Lê Thành Nam

MỤC LỤC

LỜI NÓI ĐẦU **2**

CÁCH HỌC VỚI TÀI LIỆU NÀY **3**

CHƯƠNG 1. SỐ HỌC VÀ ĐẠI SỐ NÂNG CAO **6**

Bài 1. Đếm số nguyên tố tới 10^{11} và tổng hàm nhân tính 7

Bài 2. Biến đổi trên tập con: SOS và Walsh–Hadamard 15

Bài 3. Hàm sinh 22

Bài 4. Nội suy Lagrange 28

Bài 5. Truy hồi tuyến tính: Kitamasa, Berlekamp–Massey và ma trận (min, +) 34

CHƯƠNG 2. XÁC SUẤT – TRÒ CHƠI – TỐI ƯU **41**

Bài 6. Xích Markov và khử Gauss 42

Bài 7. Trò chơi nâng cao: misère, Wythoff và phân tích ngược 48

Bài 8. Tối ưu quy hoạch động: chia để trị, Knuth và Aliens 55

CHƯƠNG 3. HÌNH HỌC TÍNH TOÁN NÂNG CAO **62**

Bài 9. Định lí Pick, phép xoay 45° và giao nửa mặt phẳng 63

Bài 10. Đường tròn nhỏ nhất, sắp xếp theo góc và tổng Minkowski 71

Lê Thành Nam

CHƯƠNG 1. SỐ HỌC VÀ ĐẠI SỐ NÂNG CAO

*Quyển 1 đã cho em sàng tuyến tính để biết mọi số tới 10^7, và kĩ thuật chia khối $\lfloor n/i \rfloor$ để tính tổng tới 10^{12} khi đã có sẵn tổng tiền tố của hàm cần dùng. Chương này đi tiếp ba hướng. Thứ nhất, đếm số nguyên tố và tính tổng hàm nhân tính tới 10^{11} khi **không** thể sàng tới n. Thứ hai, các phép biến đổi trên tập con (SOS, Walsh–Hadamard) và hàm sinh, hai ngôn ngữ biến bài toán đếm thành phép nhân. Thứ ba, nội suy Lagrange và truy hồi tuyến tính, hai cách đi thẳng tới số hạng thứ 10^{18} mà không tính các số hạng ở giữa.*

TRONG CHƯƠNG NÀY

Bài 1. Đếm số nguyên tố tới 10^{11} và tổng hàm nhân tính

Bài 2. Biến đổi trên tập con: SOS và Walsh–Hadamard

Bài 3. Hàm sinh

Bài 4. Nội suy Lagrange

Bài 5. Truy hồi tuyến tính: Kitamasa, Berlekamp–Massey và ma trận (min, +)

Lê Thành Nam

BÀI 1 Đếm số nguyên tố tới 10^{11} và tổng hàm nhân tính

Học xong bài này, em sẽ:

✓ Hiểu vì sao chỉ có khoảng $2\sqrt{n}$ giá trị khác nhau của $\lfloor n/i \rfloor$ và cách đánh chỉ số chúng bằng hai mảng.

✓ Cài đặt thuật toán Lucy đếm số nguyên tố (hoặc tính tổng các số nguyên tố) tới 10^{11} trong $O(n^{3/4}/\log n)$.

✓ Cài đặt sàng Đỗ Giáo tính tổng tiền tố của μ, φ tới 10^{10} trong $O(n^{2/3})$.

✓ Biết dùng các bảng $\pi(\lfloor n/i \rfloor)$ vừa tính để trả lời các câu hỏi đếm khác, chẳng hạn đếm số nửa nguyên tố.

KHỞI ĐỘNG

Liệt kê các giá trị $\lfloor 30/i \rfloor$ với $i = 1, 2, ..., 30$. Em sẽ được 30, 15, 10, 7, 6, 5, 4, 3, 3, 3, 2, 2, ... rồi rất nhiều số 1. Có bao nhiêu giá trị khác nhau? Chỉ 10: {1, 2, 3, 4, 5, 6, 7, 10, 15, 30}. Năm giá trị đầu là $\lfloor 30/i \rfloor$ với $i \le 5$, năm giá trị còn lại đều không vượt quá 5.

Bài học này chỉ ra rằng để đếm số nguyên tố tới 30, ta chỉ cần theo dõi đúng 10 con số này, và với $n = 10^{11}$ là khoảng 632 nghìn con số thay vì 10^{11}.

BÀI TOÁN. ĐẾM SỐ NGUYÊN TỐ [DEMNT]

Cho số nguyên dương n, hãy in $\pi(n)$, số các số nguyên tố không vượt quá n.

Dữ liệu vào:

- Một dòng chứa n ($1 \le n \le 10^{11}$).

Kết quả:

- Một số nguyên là $\pi(n)$.

Dữ liệu vào	Kết quả	Giải thích
30	10	2, 3, 5, 7, 11, 13, 17, 19, 23, 29.
100000000000	4118054813	$\pi(10^{11})$.

1. Ý tưởng ngây thơ: Sàng tới n

Sàng Eratosthenes (hoặc sàng tuyến tính) tới n rồi đếm.

Lê Thành Nam

SÀNG TỚI n TỐN BAO NHIÊU?

• Thời gian O(n log log n): khoảng $3 \cdot 10^{11}$ thao tác với $n = 10^{11}$, tức vài chục phút.

• Bộ nhớ: dù dùng một bit cho mỗi số vẫn cần $10^{11}/8$ byte ≈ 12,5 GB.

• Sàng phân đoạn giải quyết được bộ nhớ (chỉ giữ một đoạn √n số), nhưng thời gian vẫn tỉ lệ với n.

BẾ TẮC

Bất kì cách nào "chạm" vào từng số tới n đều quá chậm. Ta cần một cách đếm mà không liệt kê các số nguyên tố: chỉ làm việc với một tập nhỏ các giá trị, tính π(n) bằng truy hồi.

2. Phá bế tắc, bước một: Chỉ có khoảng 2√n giá trị ⌊n/i⌋

TẬP GIÁ TRỊ V(n)

V(n) = { ⌊n/i⌋ : 1 ≤ i ≤ n } có không quá 2√n phần tử. Với r = ⌊√n⌋: các giá trị ⌊n/i⌋ với i ≤ r, cùng các giá trị v ≤ r.

VÌ SAO?

Nếu i ≤ √n thì có không quá √n giá trị ⌊n/i⌋. Nếu i > √n thì ⌊n/i⌋ < √n, nên chỉ có không quá √n giá trị. Một tính chất quan trọng nữa: ⌊⌊n/a⌋/b⌋ = ⌊n/(ab)⌋, nên mọi phép "chia nguyên một giá trị của V(n) cho một số" đều cho lại một giá trị trong V(n).

Ta lưu một đại lượng g(v) cho mỗi v ∈ V(n) bằng hai mảng độ dài r: lo[v] = g(v) với v ≤ r, và hi[i] = g(⌊n/i⌋) với i ≤ r. Khi cần g(w) với w ∈ V(n): nếu w ≤ r thì đọc lo[w], ngược lại w = ⌊n/i⌋ với i = ⌊n/w⌋ ≤ r, đọc hi[⌊n/w⌋].

3. Phá bế tắc, bước hai: Thuật toán Lucy

Bước 1 **Đại lượng cần theo dõi**

Với mỗi v ∈ V(n) và mỗi số nguyên tố p, đặt g(v, p) = số các số x trong [2, v] mà x là số nguyên tố hoặc thừa số nguyên tố nhỏ nhất của x lớn hơn p. Nói cách khác, g(v, p) là số các số còn "sống sót" trong đoạn [2, v] sau khi sàng Eratosthenes đã gạch bội của mọi số nguyên tố không vượt quá p.

• Ban đầu (chưa gạch gì): g(v, 1) = v − 1.

• Khi $p^2 > v$, mọi hợp số không vượt quá v đều có thừa số nguyên tố nhỏ nhất ≤ √v < p, nên đã bị gạch: g(v, p) = π(v).

Lê Thành Nam

Bước 2 Truy hồi

Khi chuyển từ số nguyên tố trước p sang p, ta gạch các hợp số $x \le v$ có thừa số nguyên tố nhỏ nhất đúng bằng p. Mỗi số như vậy viết duy nhất được thành $x = p \cdot y$ với $y \le v/p$ và y "sống sót" sau các số nguyên tố nhỏ hơn p, nhưng y không phải là một số nguyên tố nhỏ hơn p.

> **TRUY HỒI LUCY**
>
> $g(v, p) = g(v, p-1) - (g(\lfloor v/p \rfloor, p-1) - g(p-1, p-1))$, áp dụng khi p nguyên tố và $p^2 \le v$. Công thức: $g(p-1, p-1) = \pi(p-1)$ là số số nguyên tố nhỏ hơn p.

Hiệu trong ngoặc đếm các $y \in [2, \lfloor v/p \rfloor]$ còn sống sót, trừ đi các y là số nguyên tố nhỏ hơn p (vì $p \cdot y$ khi đó có thừa số nhỏ nhất là $y < p$, đã bị gạch từ trước). Lưu ý $y \ge p$ tự động vì y sống sót và không phải số nguyên tố nhỏ hơn p.

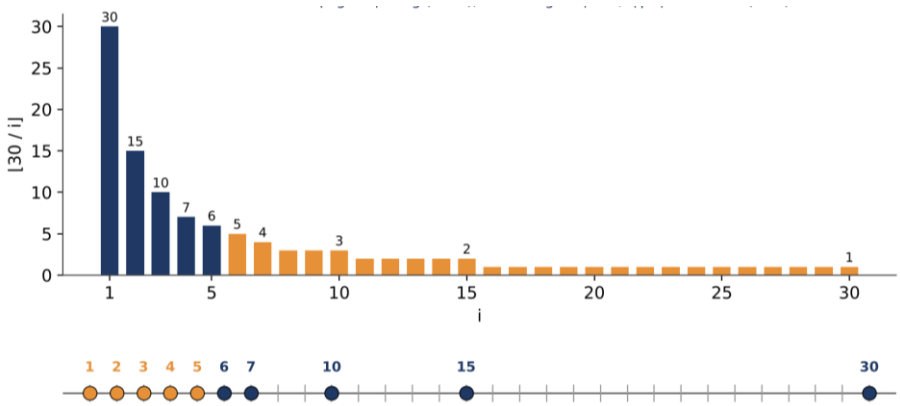
Ta không cần lưu chiều p: cập nhật mảng g tại chỗ, duyệt v từ lớn xuống nhỏ để $g(\lfloor v/p \rfloor)$ vẫn là giá trị của bước trước ($\lfloor v/p \rfloor < v$). Nhận biết p nguyên tố rất đơn giản: $lo[p] \ne lo[p-1]$.

Bước 3 Chạy tay với n = 30

$r = 5$, $V(30) = \{1, 2, 3, 4, 5, 6, 7, 10, 15, 30\}$. Ô trống nghĩa là giá trị không đổi.

v	2	3	4	5	6	7	10	15	30
ban đầu: v − 1	1	2	3	4	5	6	9	14	29
sau p = 2	–	–	2	3	3	4	5	8	15
sau p = 3	–	–	–	–	–	–	4	6	11
sau p = 5	–	–	–	–	–	–	–	–	10

Ví dụ với p = 2 ($\pi(1) = 0$): $g(30) = 29 - g(15) = 29 - 14 = 15$, rồi $g(15) = 14 - g(7) = 14 - 6 = 8$. Vì duyệt v giảm dần, khi tính g(30) thì g(15) vẫn là giá trị cũ 14. Với p = 3 ($\pi(2) = 1$): $g(30) = 15 - (g(10) - 1) = 15 - 4 = 11$. p = 4 không nguyên tố vì $lo[4] = lo[3] = 2$. Với p = 5 ($\pi(4) = 2$): chỉ v = 30 có $25 \le v$, $g(30) = 11 - (g(6) - 2) = 11 - 1 = 10$. Kết quả $\pi(30) = 10$, đồng thời ta có luôn $\pi(15) = 6$, $\pi(10) = 4$, $\pi(7) = 4$.



Hình 1.1. *Các giá trị $\lfloor n/i \rfloor$ với n = 30: dày đặc ở phần nhỏ, thưa dần ở phần lớn*

Lê Thành Nam

LUCY TỐN BAO NHIÊU?

• Với mỗi số nguyên tố $p \le \sqrt{n}$, chỉ các $v \in V(n)$ có $v \ge p^2$ bị cập nhật. Tổng số lần cập nhật là $O(n^{3/4}/\log n)$.

• Với $n = 10^{11}$: $r \approx 316\,228$, khoảng vài chục triệu phép tính, chạy chừng 0,3 giây.

• Bộ nhớ: hai mảng long long độ dài r, khoảng 5 MB.

4. Chương trình đếm số nguyên tố

DEMNT.CPP: Thuật toán Lucy, $O(n^{3/4}/\log n)$

```cpp
#include <cmath>
#include <cstdio>
#include <vector>
using namespace std;
typedef long long ll;

int main() {
    if (fopen("DEMNT.INP", "r")) {
        freopen("DEMNT.INP", "r", stdin);
        freopen("DEMNT.OUT", "w", stdout);
    }
    ll n;
    scanf("%lld", &n);
    ll r = sqrtl((long double)n);
    while (r * r > n) r--;
    while ((r + 1) * (r + 1) <= n) r++;
    // lo[v] = g(v) voi v <= r; hi[i] = g(n / i) voi i <= r
    // ban dau g(v) = v - 1 (moi so 2..v deu "con song")
    vector<ll> lo(r + 2), hi(r + 2);
    for (ll v = 1; v <= r; v++) lo[v] = v - 1;
    for (ll i = 1; i <= r; i++) hi[i] = n / i - 1;
    for (ll p = 2; p <= r; p++) {
        if (lo[p] == lo[p - 1]) continue;      // p khong nguyen to
        ll sp = lo[p - 1], p2 = p * p;
        ll lim = min(r, n / p2);
        for (ll i = 1; i <= lim; i++) {        // v = n / i, cap nhat tu lon
            ll d = i * p;
            hi[i] -= (d <= r ? hi[d] : lo[n / d]) - sp;
        }
        for (ll v = r; v >= p2; v--) lo[v] -= lo[v / p] - sp;
    }
    printf("%lld\n", hi[1]);
    return 0;
}
```

Lê Thành Nam

Vài chi tiết đáng chú ý:

• r tính bằng `sqrtl` rồi chỉnh lại bằng hai vòng while, vì căn của số 11 chữ số tính bằng số thực có thể lệch một đơn vị.

• Trong vòng cập nhật hi[i], $v = \lfloor n/i \rfloor$ và $\lfloor v/p \rfloor = \lfloor n/(i \cdot p) \rfloor$: nếu $d = i \cdot p \le r$ thì đọc hi[d], ngược lại đọc lo[⌊n/d⌋].

• Muốn tính **tổng** các số nguyên tố thay vì số lượng, chỉ cần thay "đếm" bằng "cộng": ban đầu $g(v) = 2 + 3 + \ldots + v$, và trong truy hồi nhân phần bị gạch với p (vì $x = p \cdot y$). Đây là bài 1.1 ở Tập 2.

5. Tổng tiền tố của hàm nhân tính: sàng Đỗ Giáo

Lucy trả lời câu hỏi về số nguyên tố. Với các hàm nhân tính như μ hay φ, có một công cụ khác dựa trên tích Dirichlet (xem Quyển 1, Bài 1).

Bước 1 Ý tưởng

Đặt $M(x) = \mu(1) + \mu(2) + \ldots + \mu(x)$ (hàm Mertens). Ta biết $\Sigma_{d \mid k}\, \mu(d)$ bằng 1 nếu $k = 1$ và bằng 0 nếu $k > 1$. Cộng đẳng thức này với $k = 1, 2, \ldots, x$ rồi nhóm theo $d' = k/d$:

SÀNG ĐỖ GIÁO CHO μ VÀ φ

$\Sigma_{d=1..x}\, M(\lfloor x/d \rfloor) = 1$, nên $M(x) = 1 - \Sigma_{d=2..x}\, M(\lfloor x/d \rfloor)$.

Tương tự, từ $\Sigma_{d \mid k}\, \varphi(d) = k$: $\Phi(x) = x(x+1)/2 - \Sigma_{d=2..x}\, \Phi(\lfloor x/d \rfloor)$, với $\Phi(x) = \varphi(1) + \ldots + \varphi(x)$.

VÌ SAO Σ M(⌊x/d⌋) = 1?

$\Sigma_{d=1..x}\, M(\lfloor x/d \rfloor) = \Sigma_d\, \Sigma_{e \le x/d}\, \mu(e) = \Sigma$ các cặp (d, e) với $d \cdot e \le x$ của $\mu(e)$. Nhóm theo $k = d \cdot e$: tổng bằng $\Sigma_{k \le x}\, \Sigma_{e \mid k}\, \mu(e) = 1$, vì chỉ có $k = 1$ đóng góp. Cách nhóm này dùng được cho mọi cặp hàm có tích Dirichlet đơn giản: nếu $f \ast g = h$ và biết nhanh tổng tiền tố của g và h thì tính được tổng tiền tố của f.

Bước 2 Tính nhanh

Mọi đối số xuất hiện trong đệ quy đều thuộc $V(x)$. Vế phải tính bằng chia khối (Quyển 1, Bài 3): các d có cùng $\lfloor x/d \rfloor$ được gom lại, nên một lần tính $M(x)$ tốn $O(\sqrt{x})$ nếu đã biết M tại các giá trị nhỏ hơn. Hai mẹo quyết định tốc độ:

• Ghi nhớ: mỗi giá trị của $V(n)$ chỉ tính một lần (lưu trong bảng băm hoặc mảng đánh chỉ số theo $\lfloor n/x \rfloor$).

• Sàng sẵn phần nhỏ: tính $M(x)$ cho mọi $x \le K$ bằng sàng tuyến tính. Chọn $K \approx n^{2/3}$ thì tổng thời gian là $O(n^{2/3})$.

Chạy tay $M(10)$: các $\lfloor 10/d \rfloor$ với $d = 2..10$ là 5, 3, 2, 2, 1, 1, 1, 1, 1. Với $M(5) = -2$, $M(3) = -1$, $M(2) = 0$, $M(1) = 1$, vế tổng bằng $-2 - 1 + 0 + 0 + 5 = 2$, nên $M(10) = 1 - 2 = -1$. Kiểm tra trực tiếp: $\mu(1..10) = 1, -1, -1, 0, -1, 1, -1, 0, 0, 1$, tổng -1.

SÀNG ĐỖ GIÁO TỐN BAO NHIÊU?

• Sàng tới $K = n^{2/3}$: với $n = 10^{10}$ là $K \approx 4{,}6 \cdot 10^6$, chừng 0,05 giây.

• Phần đệ quy: tổng $\Sigma\, \sqrt{(n/i)}$ trên các $i \le n/K$, cỡ $n/\sqrt{K} = n^{2/3}$.

• Đo thực tế: $M(10^{10}) = -33\,722$ trong khoảng 0,2 giây.

Lê Thành Nam

MERTENS.CPP: Sàng Đỗ Giáo tính hàm Mertens, $O(n^{2/3})$

```cpp
#include <cmath>
#include <cstdio>
#include <unordered_map>
#include <vector>
using namespace std;
typedef long long ll;
ll K;
vector<int> pre;                  // pre[i] = mu(1) + ... + mu(i), i <= K
unordered_map<ll, ll> memo;

// M(x) = 1 - sum_{d=2..x} M(x / d), gom cac d cung x / d
ll M(ll x) {
    if (x <= K) return pre[x];
    auto it = memo.find(x);
    if (it != memo.end()) return it->second;
    ll res = 1;
    for (ll d = 2; d <= x;) {
        ll q = x / d, e = x / q;
        res -= (e - d + 1) * M(q);
        d = e + 1;
    }
    return memo[x] = res;
}

int main() {
    if (fopen("MERTENS.INP", "r")) {
        freopen("MERTENS.INP", "r", stdin);
        freopen("MERTENS.OUT", "w", stdout);
    }
    ll n;
    scanf("%lld", &n);
    K = max(10LL, (ll)pow((double)n, 2.0 / 3));
    vector<int> mu(K + 1, 1), pr;
    vector<char> comp(K + 1, 0);
    for (ll i = 2; i <= K; i++) {              // sang tuyen tinh tinh mu
        if (!comp[i]) { pr.push_back(i); mu[i] = -1; }
        for (int p : pr) {
            if (i * p > K) break;
            comp[i * p] = 1;
            if (i % p == 0) { mu[i * p] = 0; break; }
            mu[i * p] = -mu[i];
        }
    }
    pre.assign(K + 1, 0);
    for (ll i = 1; i <= K; i++) pre[i] = pre[i - 1] + mu[i];
    printf("%lld\n", M(n));
    return 0;
}
```

Lê Thanh Nam

6. Mở rộng

Mở rộng 1: Đếm số nửa nguyên tố. Số $x = p \cdot q$ với $p \le q$ nguyên tố. Với mỗi $p \le \sqrt{n}$, số các q hợp lệ là $\pi(\lfloor n/p \rfloor) - \pi(p) + 1$, và mọi giá trị $\pi(\lfloor n/p \rfloor)$ đều đã có trong mảng hi của Lucy. Đây là bài 1.3.

Mở rộng 2: Tổng hàm nhân tính khác. Với Φ chỉ cần thay "1" bằng $x(x+1)/2$ trong công thức; với tổng các số nguyên tố, dùng Lucy dạng tổng. Bài 1.2 tính $\Phi(n)$ theo modulo với n tới 10^{10}.

Mở rộng 3: Thuật toán Min_25. Kết hợp bảng của Lucy (tổng f trên các số nguyên tố) với một đệ quy theo thừa số nguyên tố nhỏ nhất, ta tính được tổng tiền tố của hầu hết các hàm nhân tính có f(p) là đa thức theo p. Đây là kĩ thuật của đề chọn đội tuyển, nằm ngoài phạm vi sách.

BẪY THƯỜNG GẶP

- **Tính r = ⌊√n⌋ sai một đơn vị** do sai số số thực: phải chỉnh lại bằng phép nhân số nguyên.
- **Duyệt v tăng dần** trong bước cập nhật: g(⌊v/p⌋) khi đó đã là giá trị mới, kết quả sai.
- **Tràn số khi tính tổng các số nguyên tố:** tổng các số tới 10^{10} khoảng $5 \cdot 10^{19}$, vượt long long. Hoặc lấy dư, hoặc dùng `__int128`.
- **Quên ghi nhớ trong sàng Đỗ Giáo:** độ phức tạp bùng nổ thành $O(n^{3/4})$ hoặc tệ hơn.
- **Chọn K quá nhỏ hoặc quá lớn:** K = √n làm đệ quy chậm, K = n làm sàng chậm. Chọn $K \approx n^{2/3}$.

GHI NHỚ

- V(n) = {⌊n/i⌋} có không quá 2√n phần tử, đánh chỉ số bằng hai mảng lo và hi.
- Lucy: g(v) ← g(v) − (g(⌊v/p⌋) − π(p − 1)) với p nguyên tố, $v \geq p^2$, v duyệt giảm dần. Kết quả g(n) = π(n), và có luôn π tại mọi điểm của V(n).
- Sàng Đỗ Giáo: f $g = h \Rightarrow S_f(x) = H(x) - \Sigma_{d \geq 2}\, g(d)\, S_f(\lfloor x/d \rfloor)$, *sàng sẵn tới* $n^{2/3}$ *và ghi nhớ.*

LUYỆN TẬP

Bài	Tên	Gợi ý liên hệ
1.1	TỔNG SỐ NGUYÊN TỐ	Tổng các số nguyên tố tới 10^{10} theo modulo. Lucy dạng tổng.
1.2	TỔNG PHI	Φ(n) tới 10^{10}. Sàng Đỗ Giáo với $\Sigma_{d \mid k}\, \varphi(d) = k$.
1.3	SỐ NỬA NGUYÊN TỐ	Đếm x = p · q ≤ 10^{11}. Dùng lại mọi giá trị π(⌊n/p⌋) của Lucy.

Tự kiểm tra nhanh

1. Liệt kê V(20).

(Đáp số: {1, 2, 3, 4, 5, 6, 10, 20}, 8 phần tử.)

2. Chạy Lucy với n = 20: sau p = 2 thì g(20) bằng bao nhiêu?

(Đáp số: g(20) = 19 − g(10) = 19 − 9 = 10; sau p = 3 thì g(20) = 10 − (g(6) − 1) = 10 − 2 = 8 = π(20).)

3. Dùng sàng Đỗ Giáo tính Φ(6) từ Φ(3) = 4, Φ(2) = 2, Φ(1) = 1.

(Đáp số: 21 − (4 + 2 + 1 + 1 + 1) = 12.)

Lê Thành Nam